

Aula 14 – Técnicas de Recuperação de BD

Banco de Dados – Guia de Revisão

Contents

1 Objetivo da aula

Esta aula trata de como o SGBD recupera o banco após falhas. O foco é manter atomicidade e durabilidade mesmo quando há queda de energia, erro de sistema, falha de transação ou crash.

2 Ideia central

Se uma transação confirmou, seus efeitos devem permanecer. Se abortou ou ficou incompleta, seus efeitos não devem permanecer. A recuperação usa log, checkpoints, undo e redo.

3 Tipos de falha

- **Falha de transação:** erro lógico, violação de restrição, deadlock ou abort.
- **Falha do sistema:** queda do SGBD ou energia; memória principal é perdida.
- **Falha de mídia:** problema no disco, exigindo backup.

4 Log

O log registra operações importantes antes que o banco seja alterado permanentemente.

Registros típicos:

```
<START T1>  
<WRITE T1, X, old, new>  
<COMMIT T1>  
<ABORT T1>
```

5 Write-Ahead Logging

Regra de WAL: o registro de log deve ser gravado em armazenamento estável antes da página de dados correspondente ser gravada no banco.

Intuição: se o banco gravou uma alteração, o SGBD precisa ter informação suficiente no log para desfazer ou refazer.

6 UNDO e REDO

- **UNDO:** desfaz efeitos de transações não confirmadas.
- **REDO:** refaz efeitos de transações confirmadas que talvez não tenham chegado ao disco.

7 Exemplo visual de log

```
<START T1>  
<WRITE T1, A, 100, 80>  
<START T2>
```

```
<WRITE T2, B, 50, 70>  
<COMMIT T1>  
-- falha do sistema
```

Após a falha:

- T_1 confirmou: pode precisar de REDO.
- T_2 não confirmou: precisa de UNDO.

8 Checkpoint

Checkpoint reduz o trabalho de recuperação. Ele indica um ponto em que o SGBD sincronizou informações suficientes para não precisar analisar todo o log desde o início.

Registro típico:

```
<CHECKPOINT>
```

Com checkpoint, a recuperação procura transações ativas no checkpoint e transações iniciadas depois dele.

9 Transações confirmadas e não confirmadas

Após uma falha:

- transações com COMMIT no log devem ser preservadas;
- transações sem COMMIT devem ser desfeitas.

10 Deferred update e immediate update

10.1 Deferred update

As atualizações só chegam ao banco depois do commit. Em geral, precisa de REDO para transações confirmadas.

10.2 Immediate update

As atualizações podem chegar ao banco antes do commit. Pode exigir UNDO para transações não confirmadas e REDO para confirmadas.

11 Backup

Falha de mídia pode exigir restauração de backup e reaplicação de log.

Estratégia geral:

1. Restaurar backup.
2. Aplicar REDO de transações confirmadas desde o backup.
3. Garantir que transações não confirmadas não permaneçam.

12 Relação com ACID

- Recuperação protege **atomicidade**: transação incompleta não fica pela metade.
- Recuperação protege **durabilidade**: transação confirmada não desaparece.

13 Pegadinhas comuns

- COMMIT no log é decisivo para saber se preserva a transação.
- UNDO não é para transação confirmada.
- REDO não é para transação abortada.
- Checkpoint não é backup completo; é um ponto de controle no log.
- Falha de mídia é mais grave que falha do sistema.

14 Desafios

Desafio 1

Considere o log:

```
<START T1>
<WRITE T1, X, 10, 20>
<COMMIT T1>
<START T2>
<WRITE T2, Y, 5, 9>
-- falha
```

O que fazer com T_1 e T_2 ?

Desafio 2

Por que WAL é importante?

15 Resolução dos desafios

Desafio 1: T_1 confirmou, então deve ser preservada e pode precisar de REDO. T_2 não confirmou, então deve ser desfeita com UNDO.

Desafio 2: porque garante que o log necessário para desfazer/refazer esteja salvo antes das alterações no banco, permitindo recuperação correta.

16 Checklist final

- Sei diferenciar UNDO e REDO?
- Sei analisar log com START, WRITE, COMMIT e falha?
- Sei para que serve checkpoint?

- Sei relacionar recuperação com atomicidade e durabilidade?
- Sei diferenciar falha de transação, sistema e mídia?